NP - complete

- We are interested in optimization problem, but the complexity theory is based on **DECISION PROBLEMS**

  Response: yes / no

  why?
  - they are easier to formalize
  - it is convenient to work with binary problems
  - the complexity theory deals with each decision problem as a language

    $L \subseteq \Sigma^*$

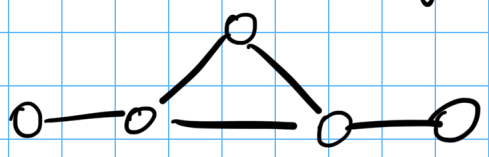
    ie, a set of strings the represent instances in which the answer is "yes"

    Ex: for the click problem: $L_{click} = \{\langle G, K\rangle \mid G$ has a click of at least size $K\}$

    thus if $\langle G, K\rangle \in L_{click}$, the answer is yes
    if $\langle G, K\rangle \notin L_{click}$, the answer is no

- In some cases, if one can resolve "efficiently" the version of decision prob. then one can solve the optimization problem by repetidly calling the solution for decision problem

- Exemple: Given $G = (V, E)$ and interge K, there exist click of size K?



- Polynomial time
  - Linear Search - n
  - binary S. - $\lg n$
  - Insertion Sort - n^2
  - Merge Sort - $n \lg n$
  - Matrix mult. - n^3
  - ⋮

- "Exponential" time
  - 0/1 Knapsack - 2^n
  - Traveling SP - 2^n
  - Sum of Subset - 2^n
  - Graph Coloring - 2^n
  - Hamiltonian cicle - 2^n
  - ⋮

- there is no known solution in poly. time for the prob.

- GUIDE / framework for doing research in decision problems
  to the name of the tool is:
  NP-Hard & NP-complete

- there exist 2 main points to talk

  ① Similarity of "exponential problems"
  ✱ the problems are related

  If one is solved, then all are solved.
  to efficiently

  ② Once we are unable to write deterministic algorithm to solve "exp. problem",
  it would be possible to write non-deterministic Alg. to in polinomial time

→ Deterministic x Non-Deterministic Algorithms

① Deterministic ⇒ we know each statement of the algorithm. and the costs are polynomial

Ex:

ⓐ by writing Merge. S. we know each routine and all costs are poly.

ⓑ by writing Alg. for Ham. Cicle in somewhere we will face a routine that takes ≥ poly.

② SearchN (A, Key)

Non-Deterministic routine

```
i ← choice()      — 1
if Key = A[i]     — 1
    Success       — 1
else
    Failure       — 1
                  ---
                  total O(1)
```

"we don't know yet how it works" ⇒ "MAGIC"

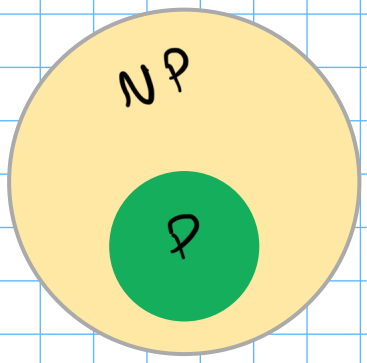
"in the future if we find, we already have a poly alg."

- Classes

  - P → Decision prob. that we have poly. deterministic Alg.

  - Nondeterministic Poly. (NP) → Decision prob. that we that we have poly. NONdeterminstic Alg.

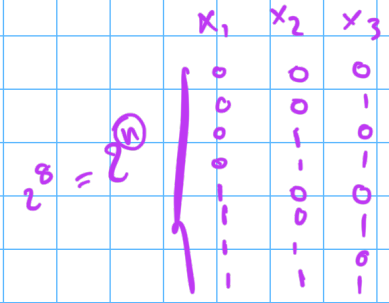
    ↳ prob. that have solutions in poly time in a NONDETERMINISTIC TURING MACHINE



- the first prob.: Satisfiability (CNF)

VAR: $X_i = \{x_1, x_2, x_3 \dots\}$

CNF: $\underbrace{(x_1 + \bar{x}_2 + x_3)}_{C_1} \cdot \underbrace{(\bar{x}_1 + x_2 + \bar{x}_3)}_{C_2}$



Prob: there exist 0/1 values for x_i that satisfy the CNF?



Stephen Cook



Leonid Levin

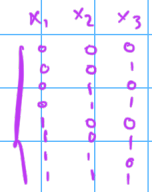
theory was published
~1960 ~1971

- How to relate SAT to others problems in "Exp. time"

  - 0/1 Knapsack

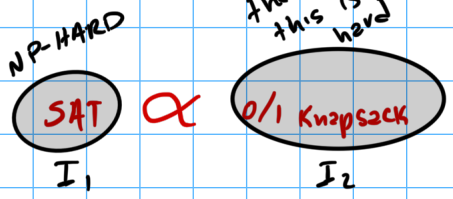
    n = 3 (objects)
    m = 8 (capacity)
    profit = {10, 8, 12}
    weights = {5, 4, 3}

    $x_i = \{0/1, 0/1, 0/1\}$



If we solve SAT, we can also solve 0/1 Knapsack

- this is the relationship

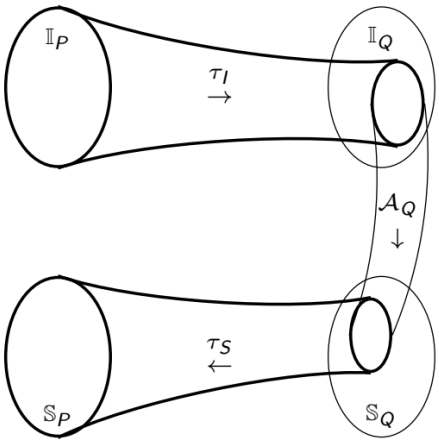


SAT $\propto L_1$ ← because hard → $L_1 \propto L_2$

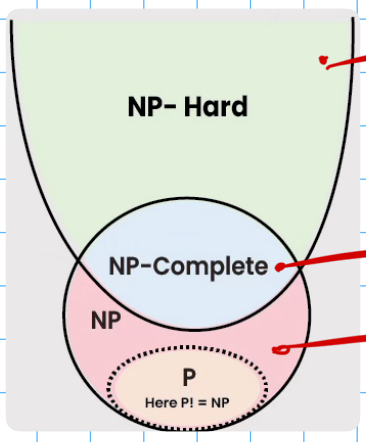
"Exponential time"
NP-Hard
- SATISFABILITY - 2^n
- 0/1 Knapsack - 2^n
- Traveling SP - 2^n
- Sum of Subset - 2^n
- Graph Coloring - 2^n
- Hamiltonian cicle - 2^n
⋮

SAT is the first prob. to be NP-Hard & NP-complete

① Problem $\in$ NP
② Any $A \in NP$, then $A \propto B$
↳ this is the def. for NP-Hard



$$\mathcal{A}_P(\mathbb{I}_P) = \tau_S(\mathcal{A}_Q(\tau_I(\mathbb{I}_P)))$$



If $\forall A \in NP$, $A \propto_p B$
" we need to show that some Known NP-HARD or NP-COMPLETE problem is reduced to B problem

① Problem $\in$ NP [$\exists$ Nondeterministic Polynomial Alg.
② be NP-HARD

LADNER THEOREM.
- If $P \neq NP$, $\exists A \in NP$ AND $\exists A \notin$ NP-complete
- If $P = NP$, ~~$\exists$~~ A in this section.
- the theorem do not show examples
- what we have are candidates

Problem	Known status
Integer factorization	It is in NP, but it is not known whether it is in P or whether it is NP-complete
Graph isomorphism	It is in NP, but it is not known whether it is in P or whether it is NP-complete
Discrete logarithm	It is in NP, but it is not known whether it is in P or whether it is NP-complete

Hypothesis	Consequence
$P \neq NP$	NP-intermediate problems exist
$P = NP$	There are no problems in NP outside P
$P = NP$	Every NP-complete problem has a polynomial-time algorithm

PART II

SAT ⇝ boolean SATISFIABILITY

boolean formula:
① VARIABLES $x_1, x_2, x_3 \ldots$ { 1/0 (Assume) as values
② they complement ($\bar{x}$), conjunction ($\cdot$ AND), Disjunction ($+$ OR)

CNF: $\underbrace{(x_1 + \bar{x}_2 + x_3)}_{C_1} \cdot \underbrace{(\bar{x}_1 + x_2 + \bar{x}_3)}_{C_2}$

Input: formula in CNF with n (→ the) variables
Prob: (or output) there exist 0/1 values for x_i that satisfy the CNF?

(the input should be a function of n meaning the input should be polynomial)

① "SAT is NP-complete" ← Cook & Levin theorem

① Any problem in NP can be solved in polynomial time by a non-deterministic RAM

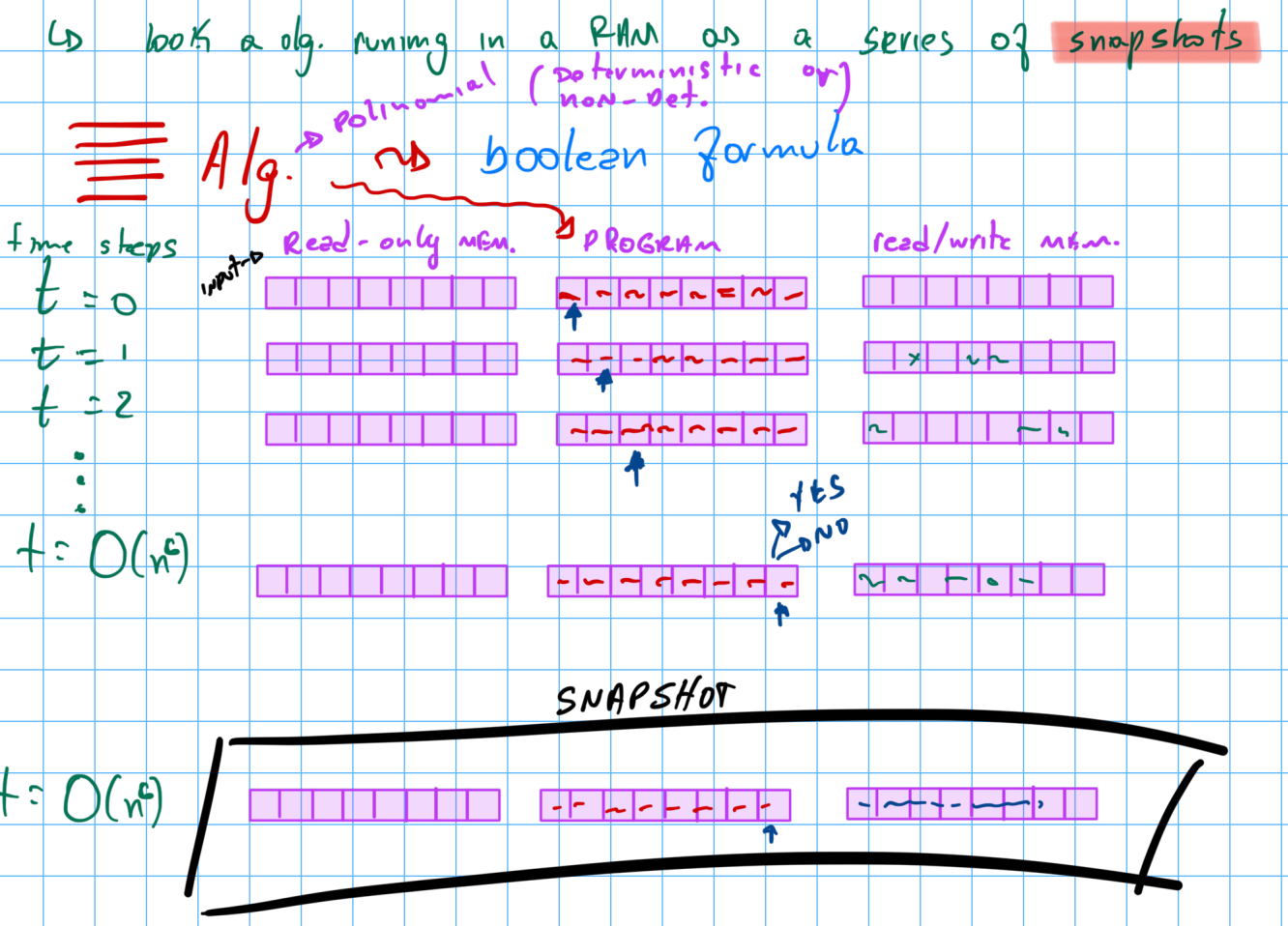
② there must be a poly. time non-deterministic Alg.

③ IDEA: Show that any algorithm can be encoded as a boolean formula

- How can we take a algorithm and transform it into a boolean formula?

Cook & Levin theorem

↳ look a alg. runing in a RAM as a series of **snapshots**



☑ EACH SNAPSHOT HAS SIZE POLY. IN N

① the size of the input is poly.

② size of the program is constant

③ in each time-step the poly. Alg. can just change a const amount of variables.

EX:

$x = a + b$

☒ THERE CAN BE AN EXPONENTIAL NUMBER OF SNAPSHOT

① the alg. is in NP to Poly.

☑ ALL SNAPSHOTS TOGETHER HAVE POLYNOMIAL SIZE

① input Poly.
+
constant
+
Poly. VARS.
=
Polynomial

- Special boolean formula

$$(x_1 + x_2 + x_3 + x_4) \cdot (\bar{x}_1 + \overline{(x_2 + x_3 + x_4)}) \cdot (\bar{x}_2 + \overline{(x_1 + x_3 + x_4)}) \cdot (\bar{x}_3 + \overline{(x_1 + x_2 + x_4)}) \cdot (\bar{x}_4 + \overline{(x_1 \; x_2 \; x_3)})$$

Quiz: What's "special" about the formula?

- [x] No satisfying assignment
- [x] Has several satisfying assignment
- [x] Can only be satisfied if exactly one variable is true
- [x] Has size $O(n^2)$ for n variables

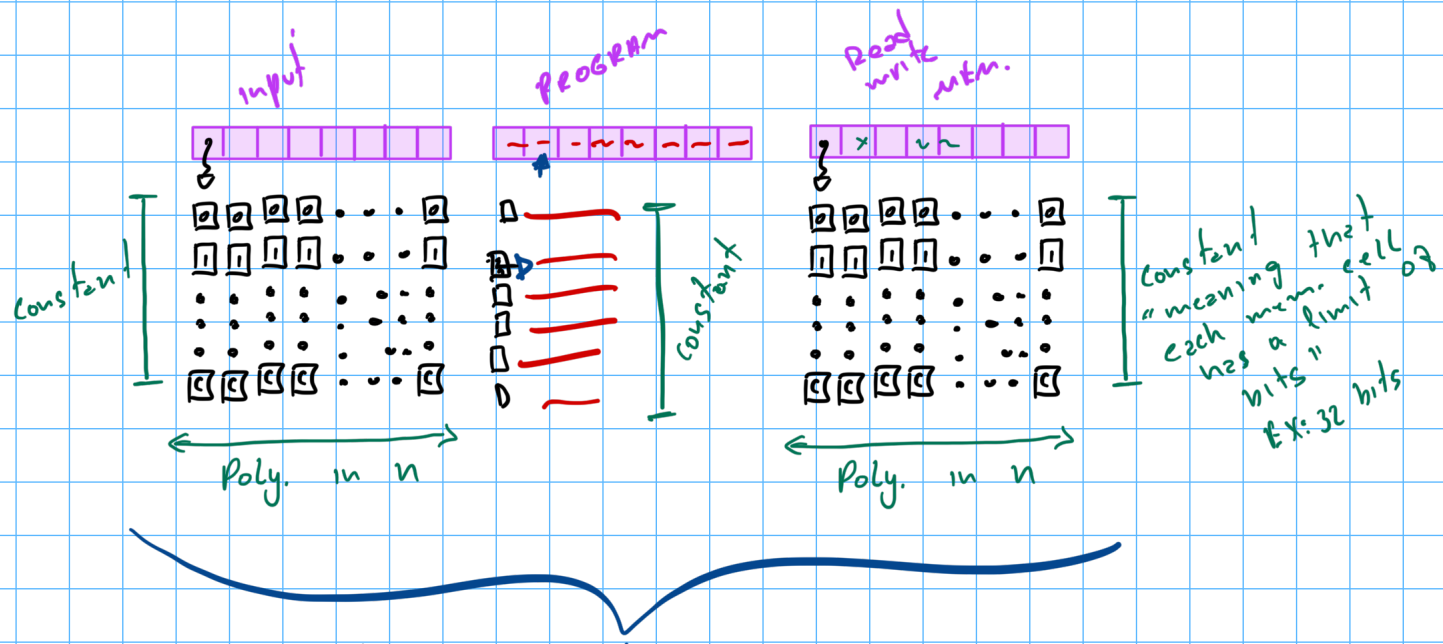
$$(x_1^{1} + x_2 + x_3 + x_4) \cdot (\bar{x}_1^{0} + \overline{(x_2 + x_3 + x_4)}) \cdot (\bar{x}_2 + \overline{(x_1^{0} + x_3 + x_4)}) \cdot (\bar{x}_3 + \overline{(x_1^{1} + x_2 + x_4)}) \cdot (\bar{x}_4 + \overline{(x_1^{1} \; x_2 \; x_3)})$$

to satisfy x_2, x_3, x_4 must be 0

$$(x_1^{1} + x_2^{0} + x_3^{0} + x_4^{0}) \cdot (\bar{x}_1^{0} + \overline{(x_2 + x_3 + x_4)}) \cdot (\bar{x}_2^{1} + \overline{(x_1^{0} + x_3 + x_4)}) \cdot (\bar{x}_3^{1} + \overline{(x_1^{1} + x_2 + x_4)}) \cdot (\bar{x}_4^{1} + \overline{(x_1^{1} \; x_2 \; x_3)})$$

$$(x_1 + x_2 + x_3 + x_4) \cdot (\bar{x}_1 + \overline{(x_2 + x_3 + x_4)}) \cdot (\bar{x}_2 + \overline{(x_1 + x_3 + x_4)}) \cdot (\bar{x}_3 + \overline{(x_1 + x_2 + x_4)}) \cdot (\bar{x}_4 + \overline{(x_1 \; x_2 \; x_3)})$$

for any given number of variables we can write a boolean formula that can be satisfied if only if exactly one variable is set to true



turn this into a boolean formule

① A variable for each black box → input mem.

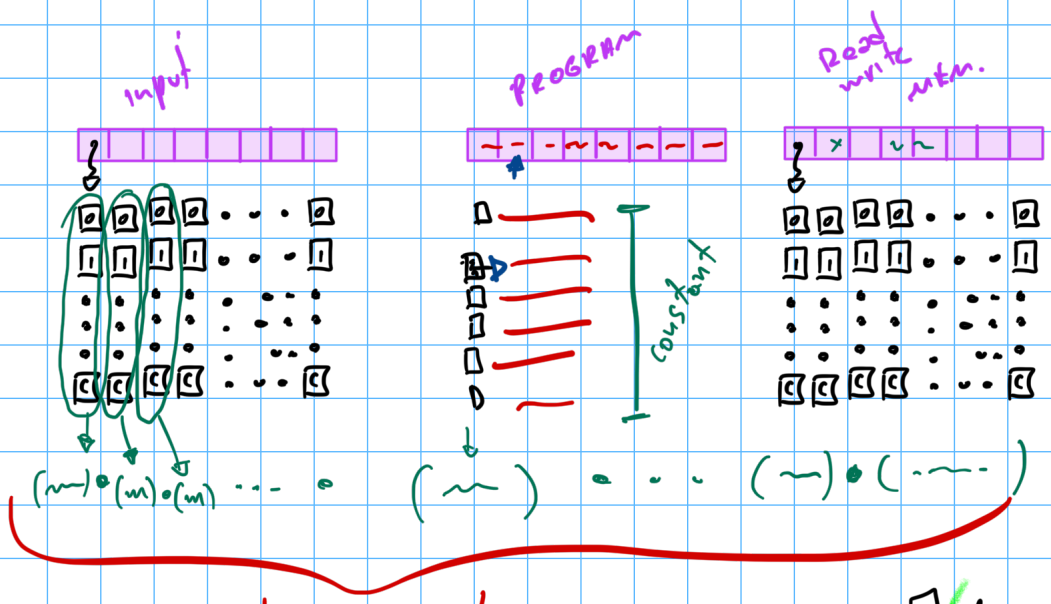
② A variable for each code inst.

③ turn each column of black box into "spetial formule" meaning that a single variable is true

③.1 it tells what goes into the mory cell as long as the formula is satisfied

④ combine formules

(next PAGE)



☑ Every satisfying assignment presents a potential SNAPSHOT

☑ Size polynomial in n

☒ only one satisfying assignment

① there exist a huge number of satisfying assignments, but each assignment ensure that only one of the variables (black box) is set to true

- A series of snapshots

① Each time-step represented a SNAPSHOT

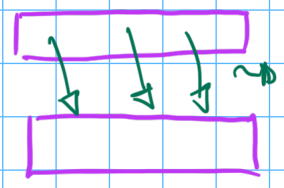
② Ensure that first SNAPSHOT represents the memory of the RAM at t=0

③ Ensure that SNAPSHOT fit together

some pages ahead

t = x

t = x+1

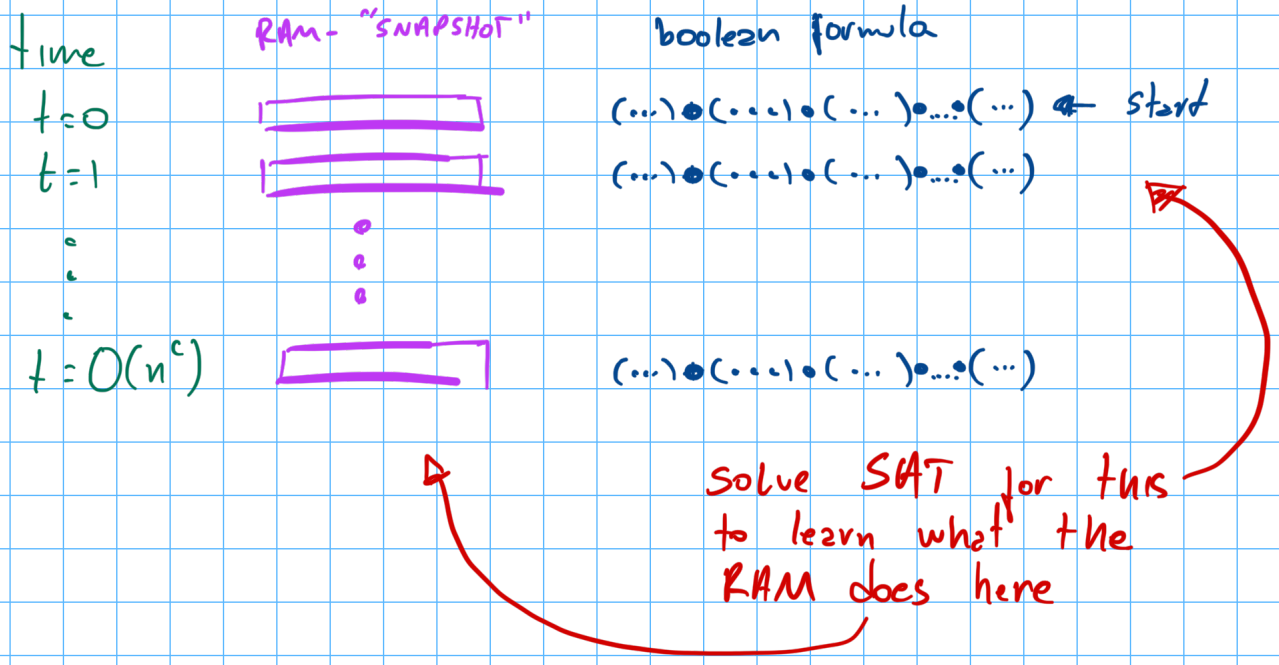


limited number of potential states that the machine can be in

① If deterministic, its nature will determine what state will be in the next point +1

② If non deterministic "choice()" there can be 2 different states

④ Ensure that boolean formula can only be satisfied if the algorithm return "yes"



Quiz: which of the following is true?

☑ Deterministic RAM: Knowing state at time t, it is clear what the state will be at time $t+1$

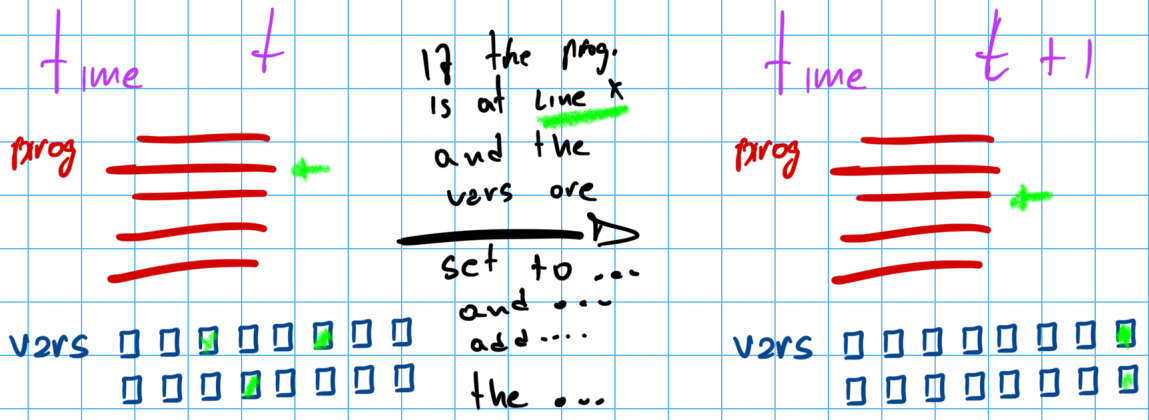
☒ what about a nondeterministic RAM( )

☑ Nondeterministic RAM behaves like a deterministic one except for the "choice()"

☒ When "choice()" is called, there can be any number of next states

① "success / fail"

- DETERMINISTIC CASE



How many Rules ?

① for single line of code, it depends on the variables
Ex: Assuming that one line of code has max of 3 vars.

② If you were to count How many rules there are

R: it's the number of different values that one variable can take times the second one times the third

③ For 1 line of code : $O(1)$
③.1 variables in RAM can't be arbitrarily large

④ the prog. has a constant number of line of code.

∴ $O(1)$

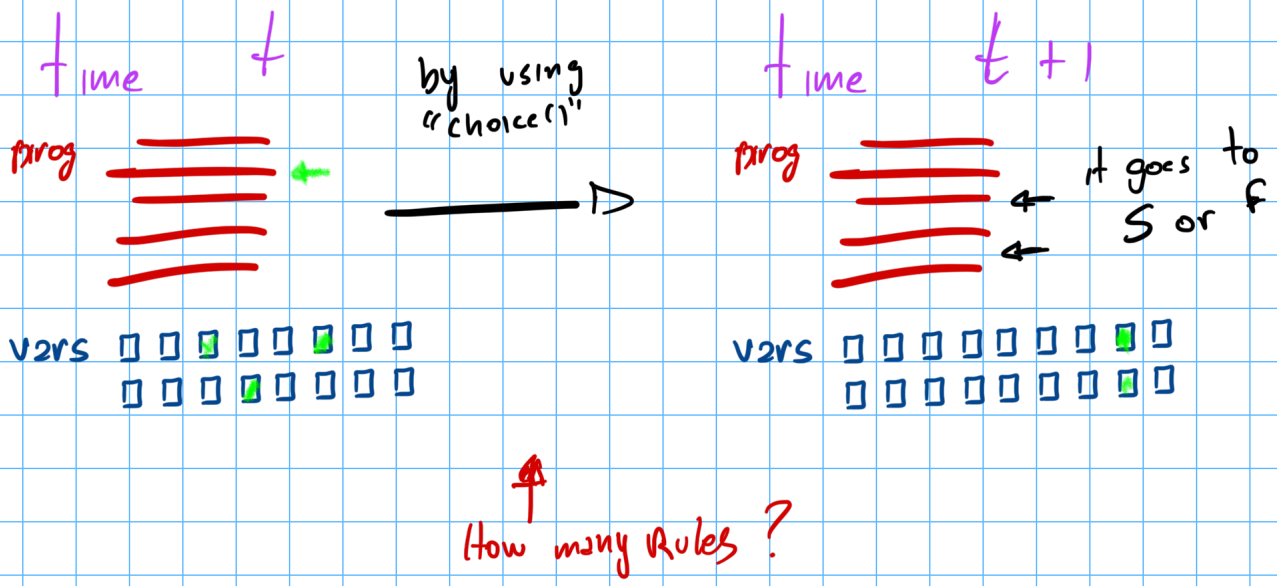
If ... then As a boolean formula

Quiz: $\overline{(x_1 \cdot x_2 \cdot x_3)} + (x_4 \cdot x_5 \cdot x_6)$

x_1	x_2	x_3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

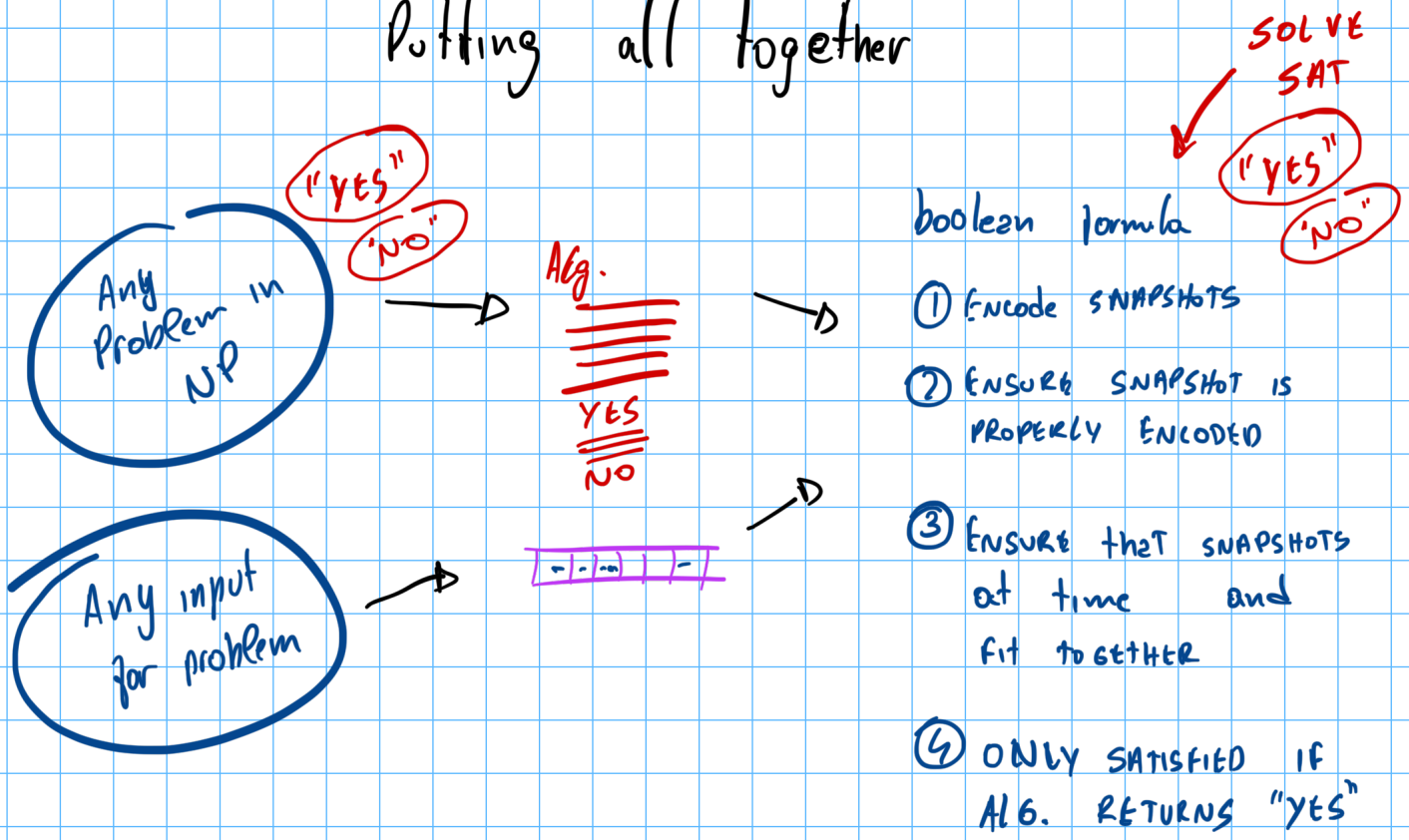
x_4	x_5	x_6	satisfied
0	0	0	☐
0	0	1	☑
0	1	0	☐
0	1	1	☒
1	0	0	☐
1	0	1	☑
1	1	0	☐
1	1	1	☑

- NON DETERMINISTIC CASE



How many Rules ?

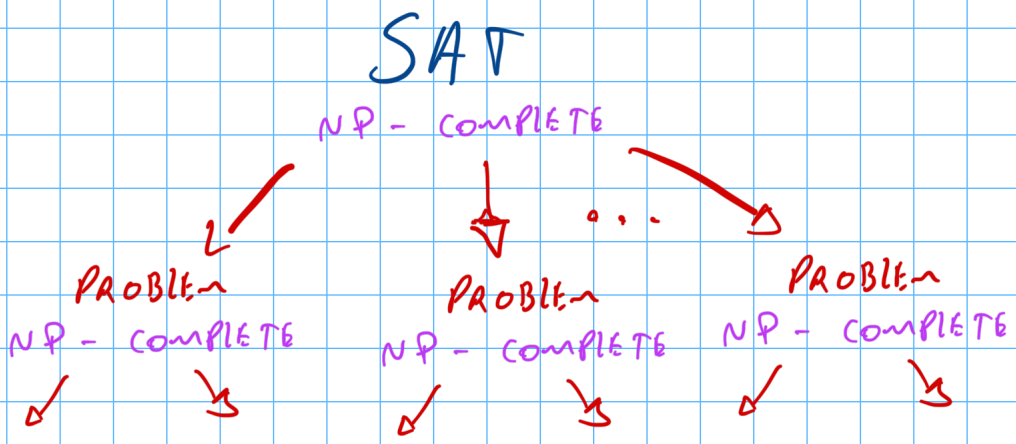
Putting all together



Quiz: Properties of the boolean formula constructed (consider input size of n)

- ☐ Constant size
- ☑ Poly size
- ☐ Exponential size
- ☑ Can encode any algorithm
- ☑ can encode any input
- ☑ Satisfiable if and only if "YES"

Any problem in NP (input size n) ⟶ SAT (size poly. in n)



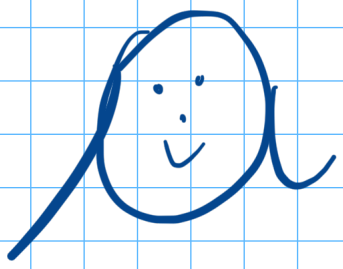
Quiz: ① what happens if you show one NP-COMPLETE problem to be solvable in poly. time? ← IN A DETERMINISTIC RAM

- ☑ Some prob. in NP can be solved in poly time
- ☑ All NP-COMPLETE problems solvable in poly. time
- ☑ All problems in NP solvable in poly time

② what happens if you show one NP-Complete problem to be solvable only in exponential time? ← IN A DETERMINISTIC RAM

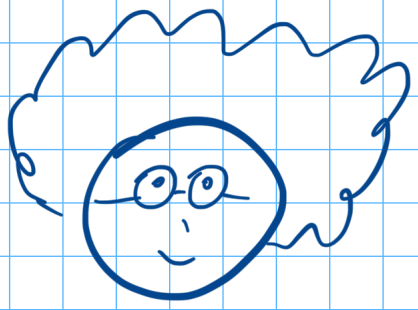
- ☑ Some prob. in NP can be solved in expo. time
- ☑ All NP-COMPLETE problems solvable in expo. time
- ☒ All problems in NP solvable in expo. time





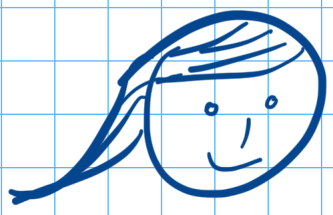
Alice

Computer networks



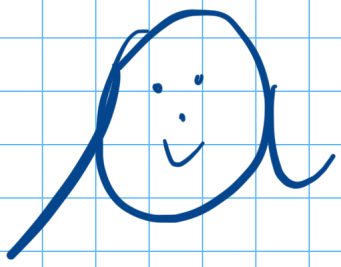
Bruno

Bio tech



Carol

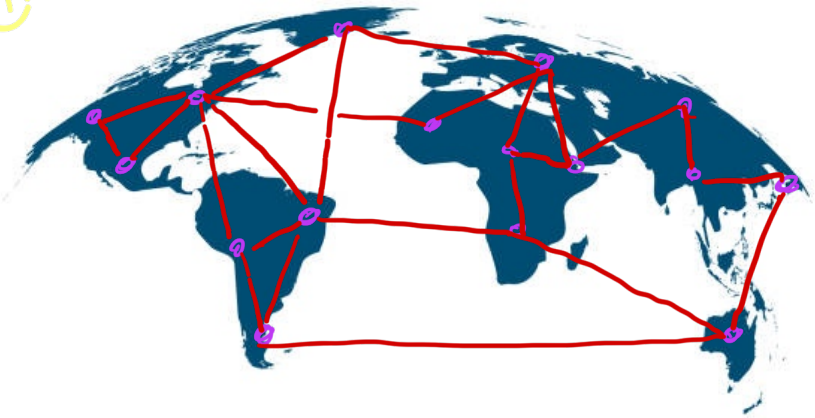
Finance



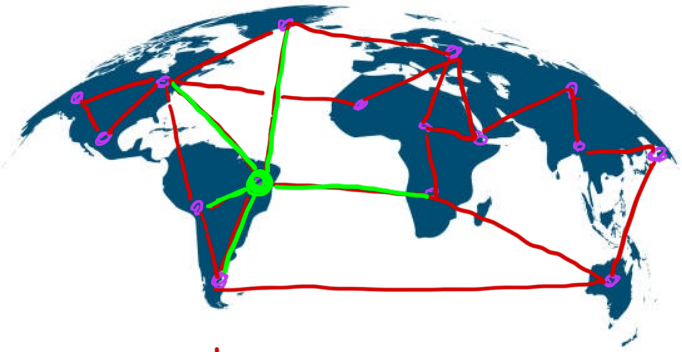
Alice

Computer networks

(1)

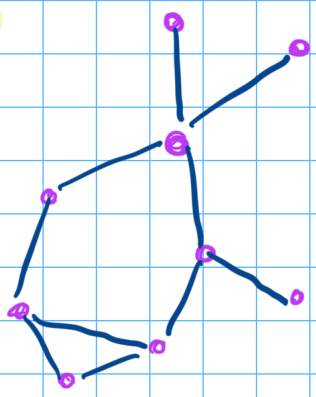


(2)



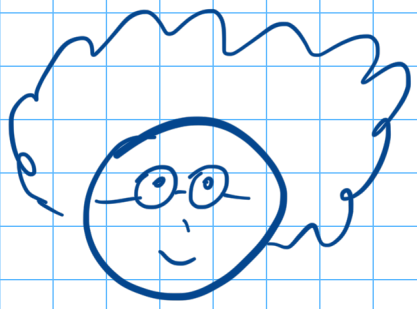
monitoring device

(3)



where to install monitoring dev. so that min number of devices is used?

4

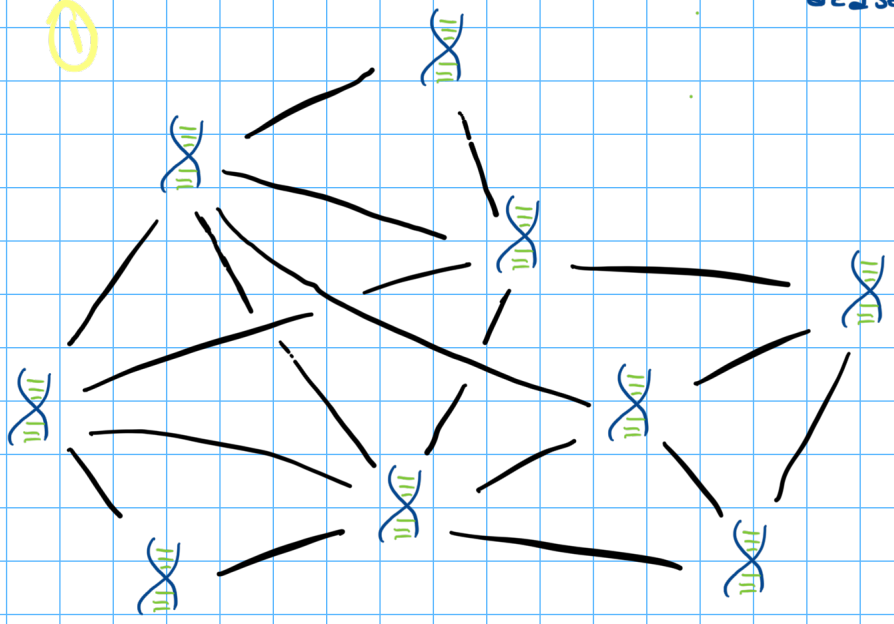


Bruno

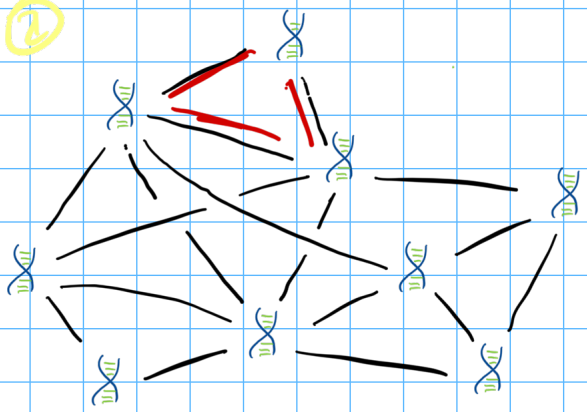
Bio tech

Bruno's work is to find group of genes that are activated when some desease occure

1

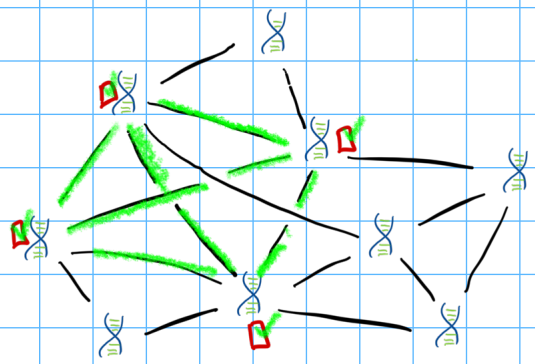


2

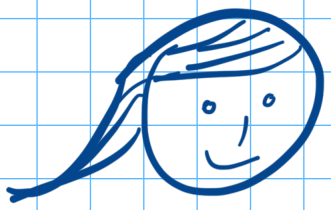


3

Quiz: what is the largest group of genes where every gene is connected to every other gene?



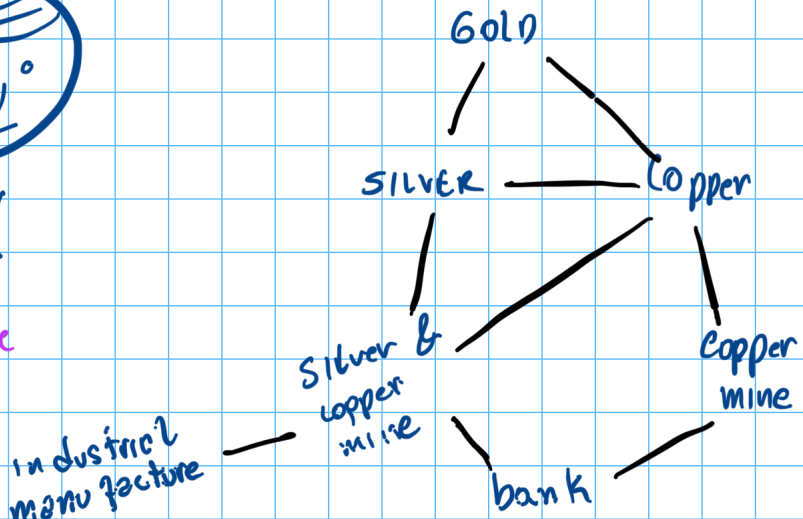
4



Carol

Finance

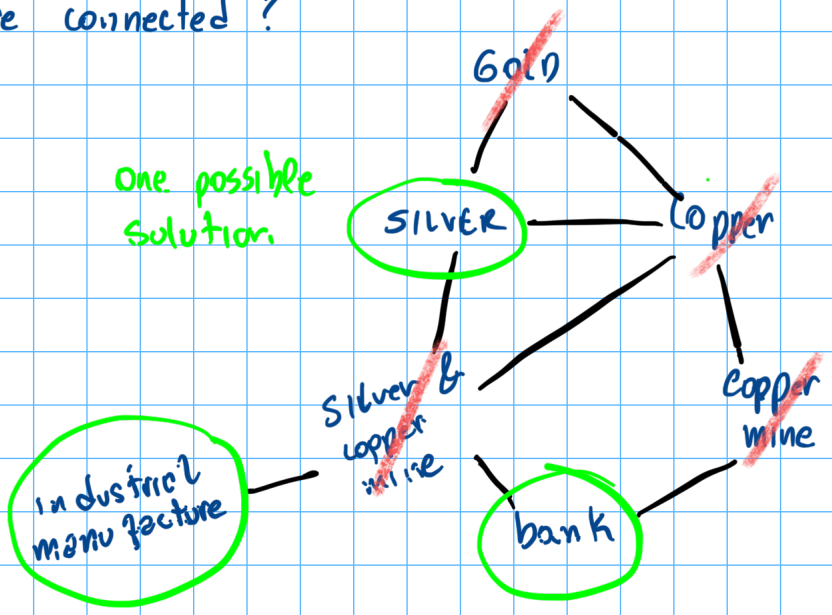
to minimize risks, she do not invest in products that are connected



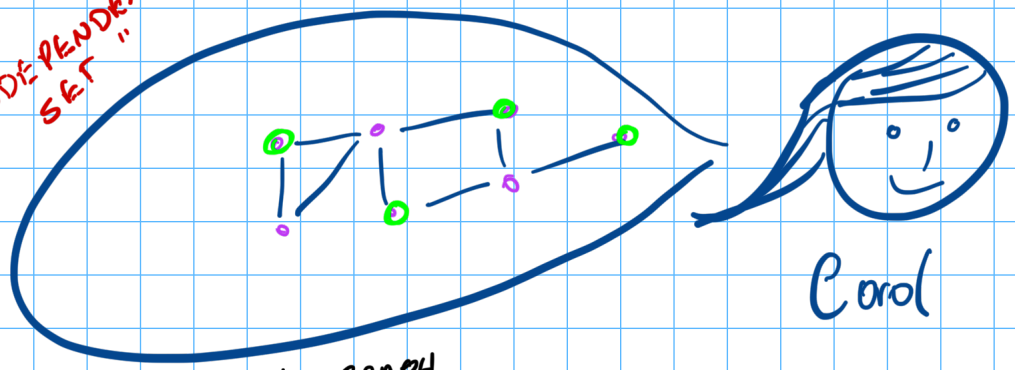
Quiz: Max. number of items that carol can select so that no one are connected?

3

One possible solution.



"INDEPENDENT SET"

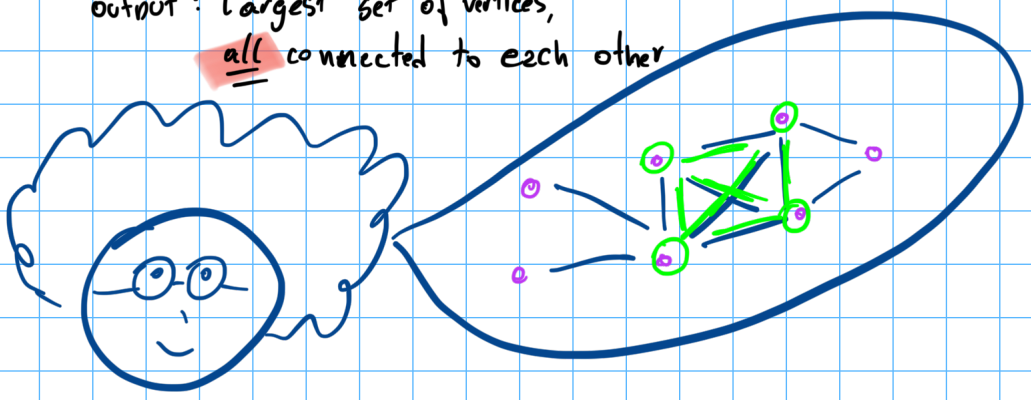


Carol

Input: GRAPH
output: largest set of vertices,
none connected to each other

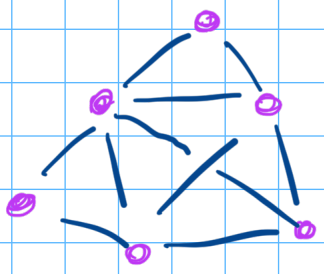
Input: GRAPH
output: largest set of vertices,
all connected to each other

"CLIQUE"

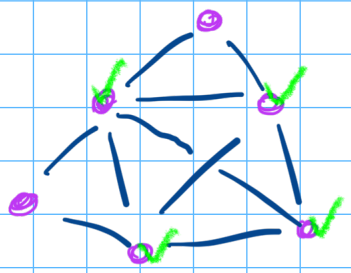


Bruno

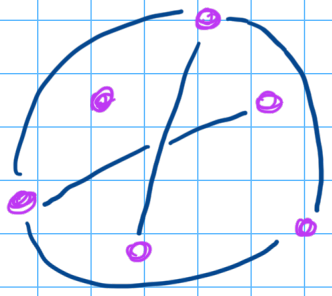
Clique



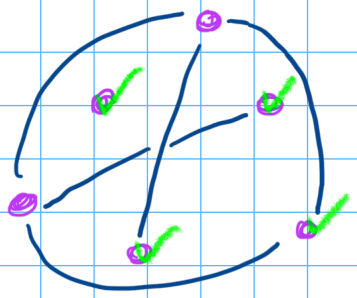
Quiz: which vertices belong to the largest "clique"



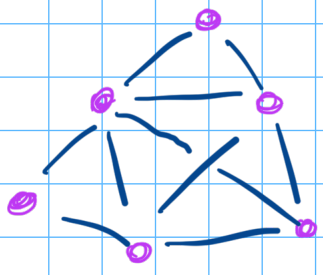
Independent Set



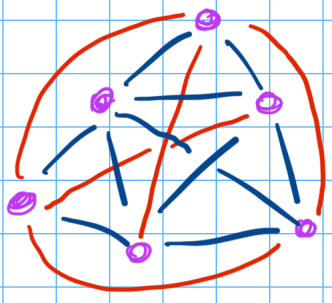
Quiz: which vertices belong to the largest "Ind. Set"



Clique



(1)

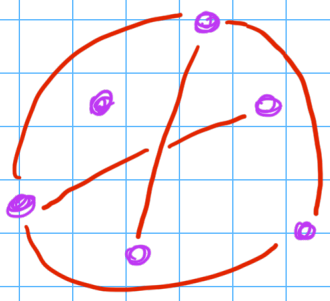


(2)



Graph
↓
inverse graph
↓
solve Inde. Set

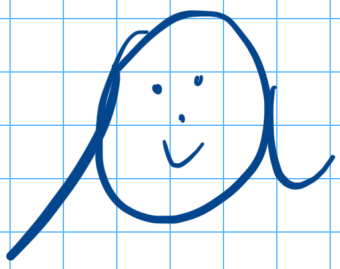
Independent Set



(3)

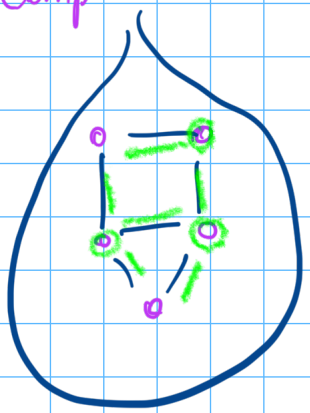


Graph
↓
inverse graph
↓
solve Clique



Alice

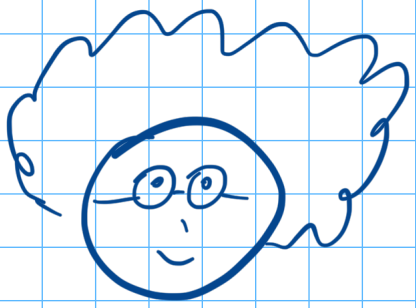
Computer networks



"VERTEX COVER"

Input: GRAPH

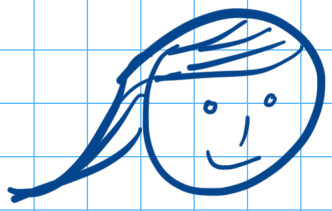
output: smallest set of vertices so that all edges are covered



Bruno

Bio tech

"CLIQUE"

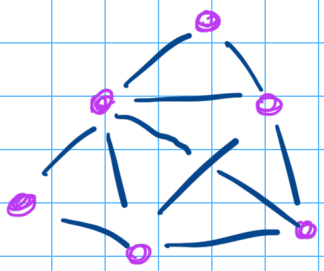


Carol

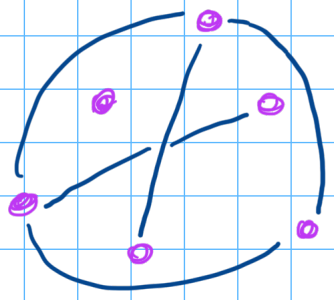
Finance

"IND. SET"

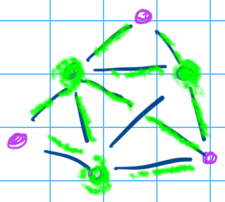
CLIQUE



INDEPENDENT SET

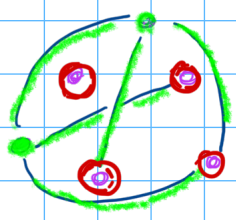


Quiz: size of minimun VERTEX COVER of each graph

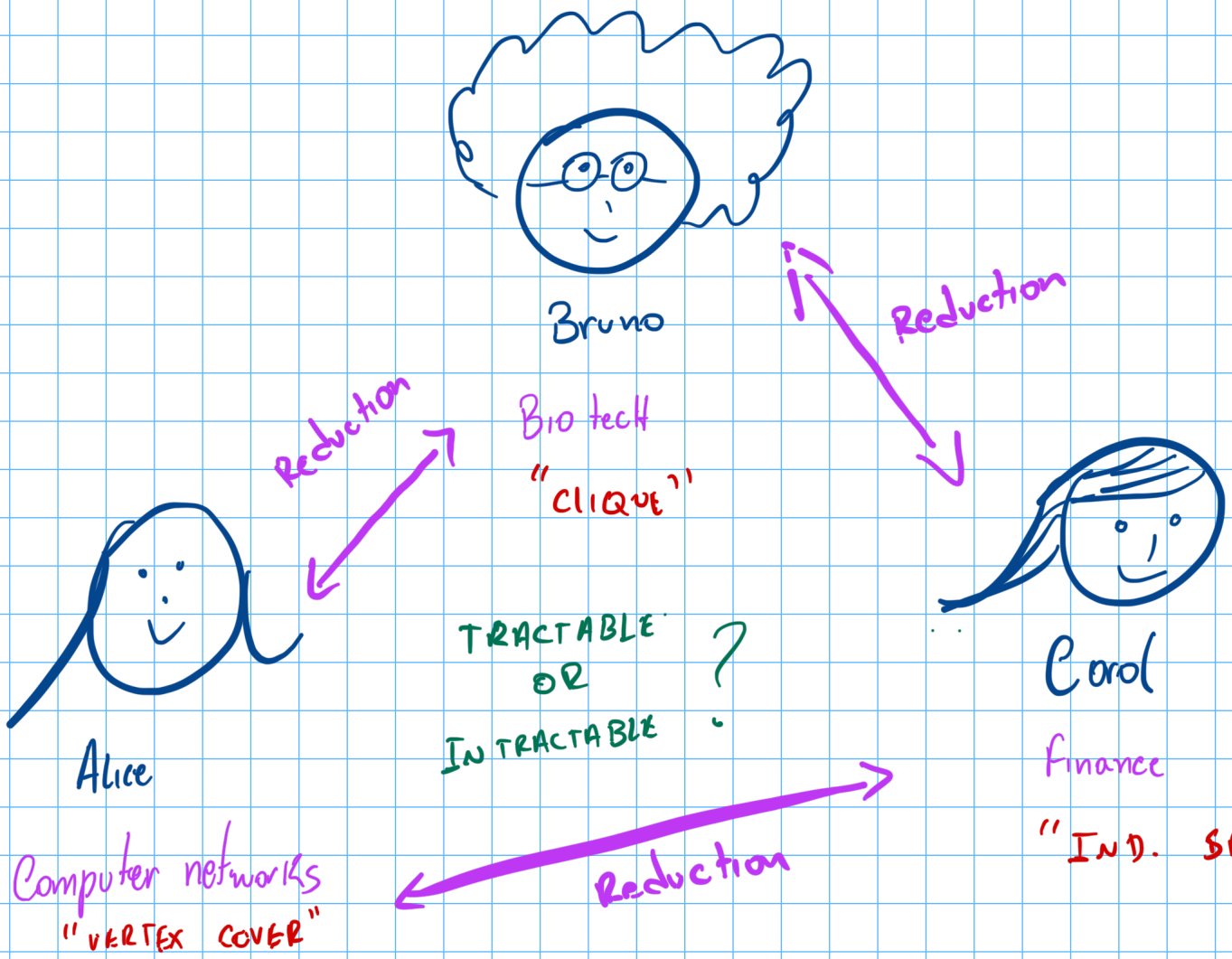


3

2



Vertex cover is the opposite of IND. Set


Bruno
Bio tech
"CLIQUE"
Reduction
Reduction
TRACTABLE
OR
INTRACTABLE
?
Alice
Computer networks
"VERTEX COVER"
Carol
finance
"IND. SET"
Reduction